

DistributedDataAccessLab

V3 - Third Deliverable

E-Commerce Marketplace - Blazor Frontends

Alexandre Schneider

FDU MSACS - CSCI 6844 V1
Programming for the Internet

April 2026

V3 - Third Deliverable Overview

This deliverable adds two Blazor Server frontends to the existing microservices backend. Both frontends run as Docker containers alongside the 6 backend services, RabbitMQ, and the API Gateway, bringing the total to 10 containers managed by docker-compose.

Complete Service Map (10 containers)

Service	Port	Description
API Gateway (YARP)	:5000	Reverse proxy routing + aggregated endpoint
CustomerService	:5001	Buyers/sellers management, role-based dashboards
OrderService	:5002	Order creation, status transitions, RabbitMQ publisher
ProductService	:5003	Product catalog, stock, discounts
CartService	:5005	Shopping cart with checkout orchestration
PaymentService	:5006	Payment processing (mock 90% approval)
NotificationService	:5007	Notifications + RabbitMQ consumers
RabbitMQ	:5672/:15672	Message broker (management UI on 15672)
SimpleFrontend	:5010	Simple Blazor dashboard with data tables
WebFrontend	:5020	AI-generated FDU-themed e-commerce marketplace

A) SimpleFrontend (port 5010)

Objective: Minimal Blazor Server frontend using the default project template with Bootstrap tables.

A straightforward dashboard that calls the API Gateway and displays raw data from every microservice. No custom styling - uses the default Blazor template with standard Bootstrap classes. The goal is to demonstrate the frontend consuming all backend endpoints via IHttpClientFactory.

Project Structure

File	Description
SimpleFrontend.csproj	.NET 10 Blazor Server project (default template)
Program.cs	Registers IHttpClientFactory with named 'Api' client pointing to API Gateway
Dockerfile	Multi-stage build (sdk:10.0-alpine -> aspnet:10.0-alpine)
Components/Layout/NavMenu.razor	Navigation with 7 links: Home, Customers, Products, Orders, Cart, Payments, Notifications

Blazor Pages

Page	Route	Functionality
Home.razor	/	Dashboard: service health table with OK/Error badges and record counts
Customers.razor	/customers	Auto-loaded table: ID, Name, Email, Phone, Role (badge), Address, Active, Created
Products.razor	/products	Auto-loaded table: ID, Seller, Name, Category, Price, Stock, Discount%, Final Price
Orders.razor	/orders	Order table + Create Order form (Buyer ID, Product ID, Qty) - triggers RabbitMQ event

Page	Route	Functionality
CartPage.razor	/cart	Lookup by Buyer ID - cart items table + Add Item form
Payments.razor	/payments	Auto-loaded table: ID, Order, Buyer, Amount, Method, Status (color badge), timestamps
Notifications.razor	/notifications	Lookup by User ID - notifications table showing RabbitMQ-generated entries

Key Technical Details

- Uses IHttpConnectionFactory with named client 'Api' - base URL configurable via Services__ApiGateway env var
- All pages use @rendermode InteractiveServer for real-time interactivity
- Create Order form on Orders page publishes OrderCreatedEvent via RabbitMQ (through the backend)
- Notification page shows entries created by RabbitMQ consumers in real-time
- Docker: runs on port 5010, depends_on apigateway

B) WebFrontend (port 5020) - AI-Generated FDU Course Marketplace

Objective: Fully styled Blazor e-commerce frontend built with AI-assisted web scraping and code generation.

How it was built

1. Web Scraping: The FDU website (fdu.edu) was scraped to capture the university's visual identity - colors, typography, layout patterns, logo usage, and overall design language.
2. AI-Generated CSS/HTML: Using the scraped reference material, AI (GitHub Copilot) generated all the custom CSS, HTML structure, component layouts, and responsive design from scratch. No FDU assets, images, or copyrighted content were copied; only the visual style was used as inspiration.
3. E-Commerce Course Marketplace: The concept is a course-selling platform where FDU sellers list courses/products and buyers can browse, add to cart, place orders, and receive notifications.

Project Structure

File/Folder	Description
WebFrontend.csproj	.NET 10 Blazor Server project
Program.cs	6 typed HttpClient services (one per microservice, direct calls) + ProfileState
Models/	Customer.cs, Product.cs, Order.cs, Cart.cs, Payment.cs, Notification.cs
Services/	CustomerService.cs, ProductService.cs, OrderService.cs, CartService.cs, PaymentService.cs, NotificationService.cs, ProfileState.cs
wwwroot/app.css	FDU-inspired custom CSS: maroon/burgundy primary, gold accents, dark tones
Components/Layout/	MainLayout.razor (custom styled), NavMenu.razor (role-based navigation)
Dockerfile	Multi-stage build (same pattern as other services)

Pages

Page	Route	Functionality
------	-------	---------------

Page	Route	Functionality
Home.razor	/	Landing page with FDU-themed hero section and marketplace overview
Dashboard.razor	/dashboard	Role-based dashboard (Admin/Buyer/Seller) with stats and quick actions
Customers.razor	/customers	Customer management with role badges and detail views
Products.razor	/products	Product catalog with seller info, discounts, stock management
Orders.razor	/orders	Order management with status transitions and cancel/return flows
CartPage.razor	/cart	Shopping cart with checkout orchestration
Payments.razor	/payments	Payment history with status badges and processing
Notifications.razor	/notifications	Notification center with read/unread management

Key Features

- FDU-inspired color scheme: CSS variables with maroon/burgundy primary, gold accents, dark backgrounds
- Role-based ProfileState: Admin sees everything, Buyer sees own orders/cart, Seller sees own products/sales
- Typed HttpClient services per microservice (CustomerService, ProductService, etc.) - NO API Gateway dependency
- Full CRUD operations on all pages with forms and validation
- Responsive layout with Blazor CSS isolation (.razor.css files)
- Docker: runs on port 5020, env vars configure direct service URLs

C) Docker Compose Updates

Two new services added to docker-compose.yml:

Service	Configuration
simplefrontend	Build: SimpleFrontend/SimpleFrontend, Port: 5010->8080, env: Services__ApiGateway=http://apigateway:8080, depends_on: apigateway
webfrontend	Build: WebFrontend/WebFrontend, Port: 5020->8080, env: 6 service URLs (direct), depends_on: apigateway

Total containers: 10 (6 microservices + RabbitMQ + API Gateway + SimpleFrontend + WebFrontend)

D) Other Changes

File	Change
.gitignore	Removed "WebFrontend/" exclusion to include it in the repository
DistributedDataAccessLab.sln	Added SimpleFrontend project (WebFrontend was already in solution)
README.md	Updated service table (10 services) + added V3 Plus section with AI explanation



AI as a Development Accelerator

4